

Министерство образования Республики Беларусь
Учреждение образования
Белорусский государственный университет информатики и
Радиоэлектроники

Факультет информационных технологий и управления
Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №1
по курсу “Логические основы интеллектуальных систем”
Вариант 3

Выполнил:
Студент гр. 421702

Евик А.Н.

Проверил:

Ивашенко В.П.

Минск

2026

Содержание

1. Грамматика сокращенного языка логики высказываний.....	3
2. Описание реализации.....	4
2.1. Структуры данных.....	4
2.2. Алгоритм.....	4
3. Схемы алгоритма.....	6
3.1 Схема алгоритма разбора строки.....	6
4. Тестирование программы.....	12
Вывод.....	14
Список использованных источников.....	15
Приложение А.....	16

Тема:

Программирование операций обработки и преобразований формул языка логики высказываний.

Цель:

Приобрести навыки программирования алгоритмов синтаксического разбора, интерпретаций и преобразований формул языка логики высказываний.

Задание:

Определить, является ли формула сокращенного языка логики высказываний невыполнимой.

Формула называется невыполнимой логической формулой тогда и только тогда, когда она обозначает функцию, равнозначную константе $\perp$.

1. Грамматика сокращенного языка логики высказываний

<логическая константа> ::= 1|0

<символ> ::= A|B|C|D|E|F|G|H|I|J|K|L|M|N|O|P|Q|R|S|T|U|V|W|X|Y|Z

<отрицание> ::= !

<конъюнкция> ::= $\wedge$

<дизъюнкция> ::= $\vee$

<импликация> ::= $\rightarrow$

<эквиваленция> ::= $\sim$

<открывающая скобка> ::= (

<закрывающая скобка> ::=)

<атомарная формула> ::= <латинская заглавная буква>

<унарная сложная формула> ::= <открывающая
скобка><отрицание><формула><закрывающая скобка>

$\langle \text{бинарная сложная формула} \rangle ::= \langle \text{открывающая скобка} \rangle \langle \text{формула} \rangle$

$\langle \text{бинарная связка} \rangle \langle \text{формула} \rangle \langle \text{закрывающая скобка} \rangle$

$\langle \text{сложная формула} \rangle ::= \langle \text{унарная сложная формула} \rangle \mid \langle \text{бинарная сложная формула} \rangle$

$\langle \text{формула} \rangle ::= \langle \text{логическая константа} \rangle \mid \langle \text{атомарная формула} \rangle \mid \langle \text{сложная формула} \rangle$

2. Описание реализации

Для реализации был выбран язык C++ стандарта C++17.

2.1. Структуры данных

Для хранения структуры логической формулы использовалось бинарное несбалансированное дерево. Каждый узел дерева содержит в себе вид узла: (переменная, константа, отрицание, конъюнкция, дизъюнкция, импликация, эквиваленция), исходный фрагмент текста формулы, значение для константы, имя для переменной, индекс переменной в таблице истинности, а также указатели на левое и правое поддереву. В методах использовались структуры данных стандартной библиотеки: умные указатели для владения узлами дерева, ассоциативный массив для хранения интерпретации и индексов переменных, упорядоченное множество для накопления уникальных переменных, динамический массив и строка.

2.2. Алгоритм

Алгоритм начинает работу после передачи в метод `Parser::parse()` строки в качестве аргумента. Разбор строки происходит методом рекурсивного спуска и реализован в методе `Parser::parse_formula()` следующим образом:

- Чтение текущего символа исходной строки.
- Если символ является логической константой (0 или 1) - создание листового узла соответствующего вида.
- Если символ является заглавной латинской буквой - создание листового узла переменной, добавление переменной в множество уникальных

переменных и присвоение ей порядкового индекса при первом появлении.

- Если символ является открывающей скобкой - определение типа сложной формулы:
 - Если следующий символ - отрицание (!), то выполняется рекурсивный разбор операнда и создается узел отрицания.
 - Иначе выполняется рекурсивный разбор левого операнда, затем чтение бинарного оператора ($\wedge$, $\vee$, $\rightarrow$, $\sim$), затем рекурсивный разбор правого операнда и создание узла соответствующего вида.
- Проверка наличия закрывающей скобки и сохранение исходного фрагмента текста в поле `text` созданного узла.

Для проверки логического выражения на выполнимость объект класса `Node`, представляющий корень дерева, передается в функцию построения таблицы истинности вместе с множеством переменных формулы.

Проверка происходит следующим образом:

- Формирование упорядоченного списка переменных из множества уникальных переменных формулы.
- Подсчёт количества строк таблицы истинности как 2 в степени количества переменных.
- Перебор всех возможных интерпретаций переменных в виде последовательного увеличения битовой маски от 0 до $2^n - 1$, где каждый бит маски соответствует значению одной переменной.
- Для каждой интерпретации - заполнение ассоциативного массива значений переменных и вычисление значения формулы методом `Node::eval()`, который рекурсивно обходит дерево и применяет операции в соответствии с видом каждого узла.
- Если на текущей интерпретации значение формулы равно истине, формула признается выполнимой и дальнейший перебор прекращается.
- Если ни одна интерпретация не дала значения «истина», формула признается невыполнимой.

3. Схемы алгоритма

3.1 Схема алгоритма разбора строки

Схема алгоритма поделена на отдельные схемы отдельных функций для большей читаемости.

Функция `Node::eval()` определяет тип логической связки:

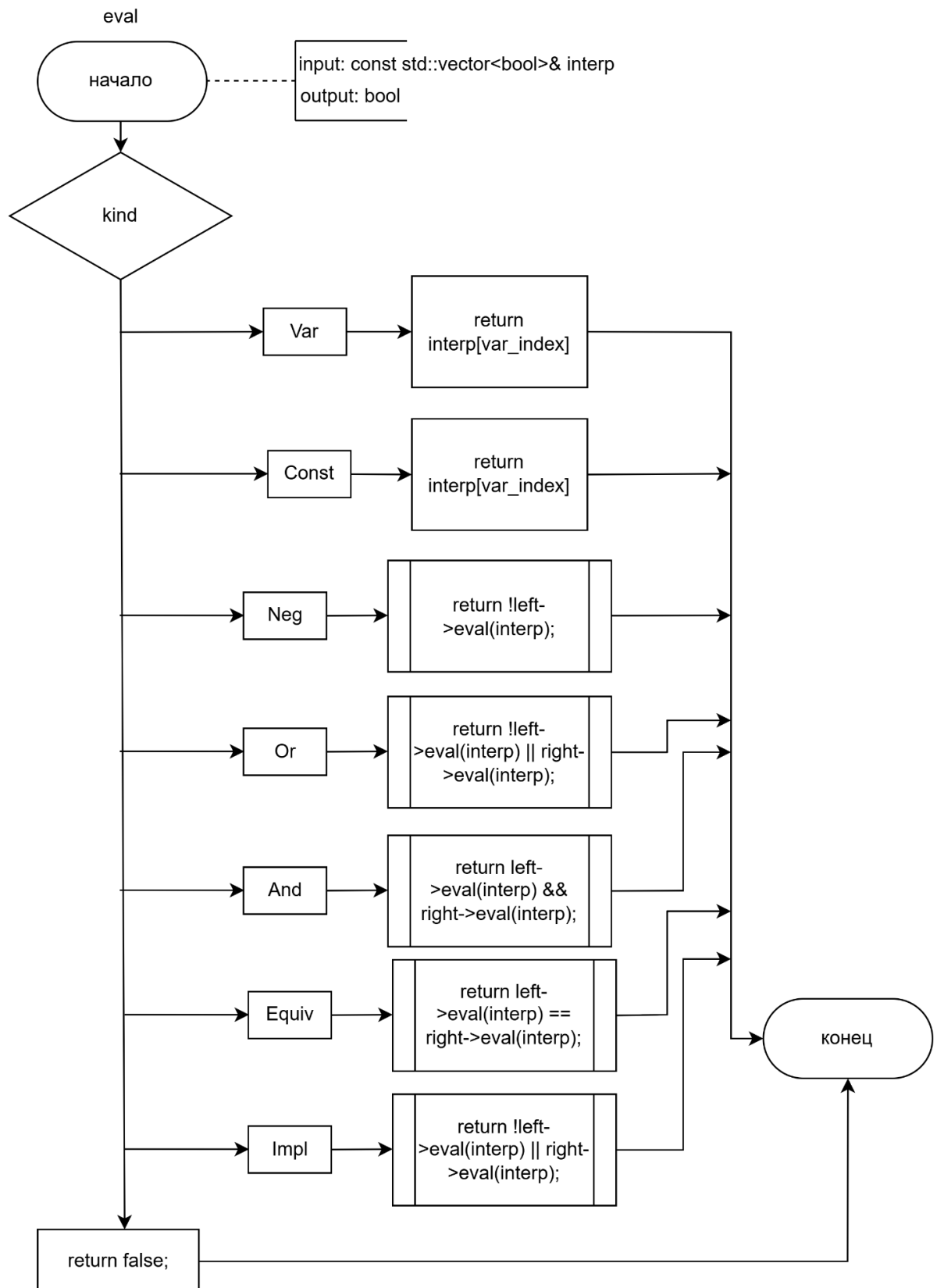


Рис. 3.1 - Node::eval()

Функция Parser::current() позволяет обратиться к текущему символу строки:

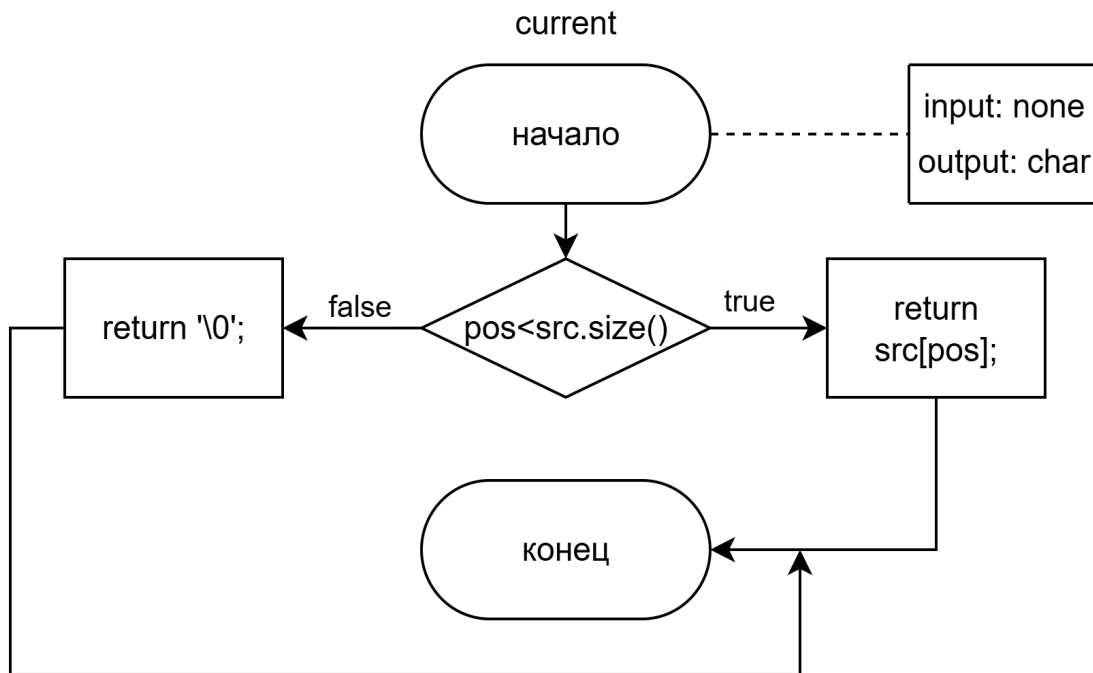


Рис. 3.2 - `Parser::current()`

Функция `Paser::move_symbol()` позволяет сдвинуться на символ вперед по строке:

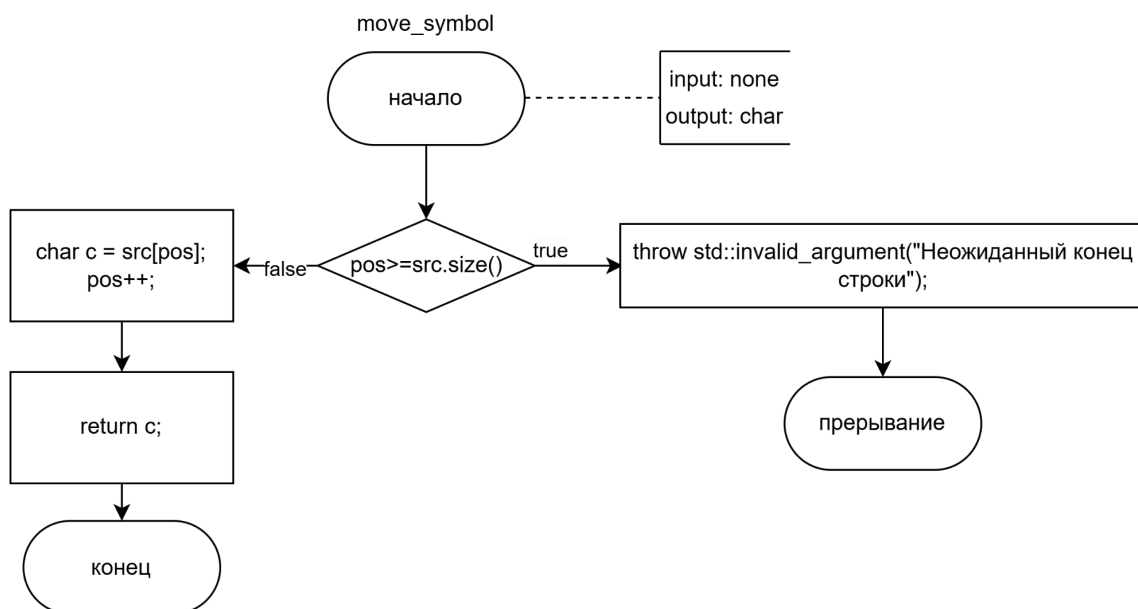


Рис. 3.3 - `Parser::move_symbol()`

Функция `Parser::expect()` ожидает на выходе определенный символ, если он не совпадает с ожидаемым - выбрасывается ошибка:

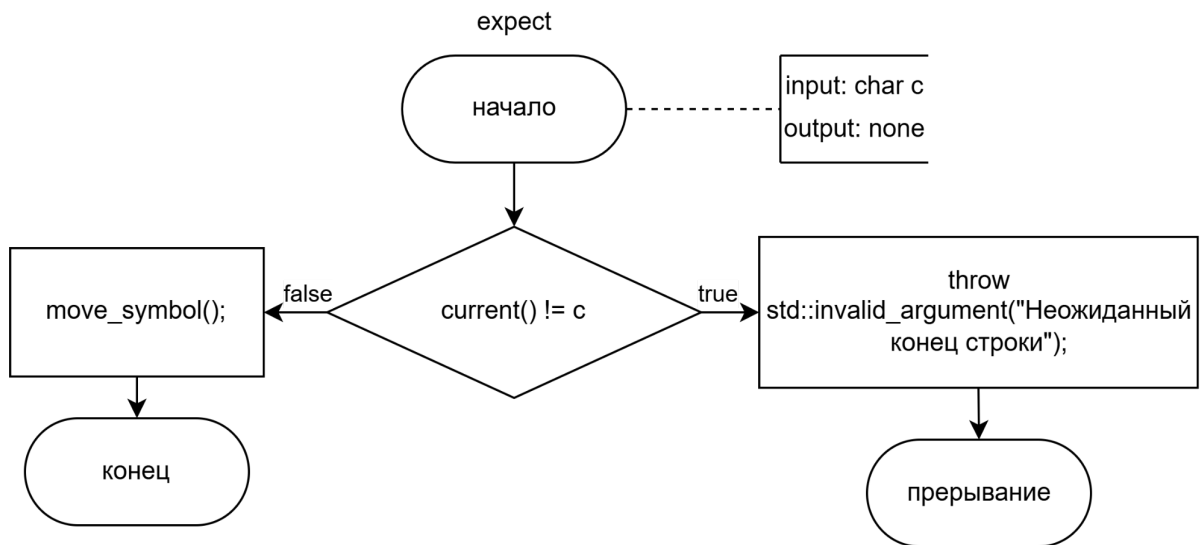


Рис. 3.4 - Parser::expect()

Функция `Parser::parse()` реализует обработку формулы сокращенного языка логики высказываний:

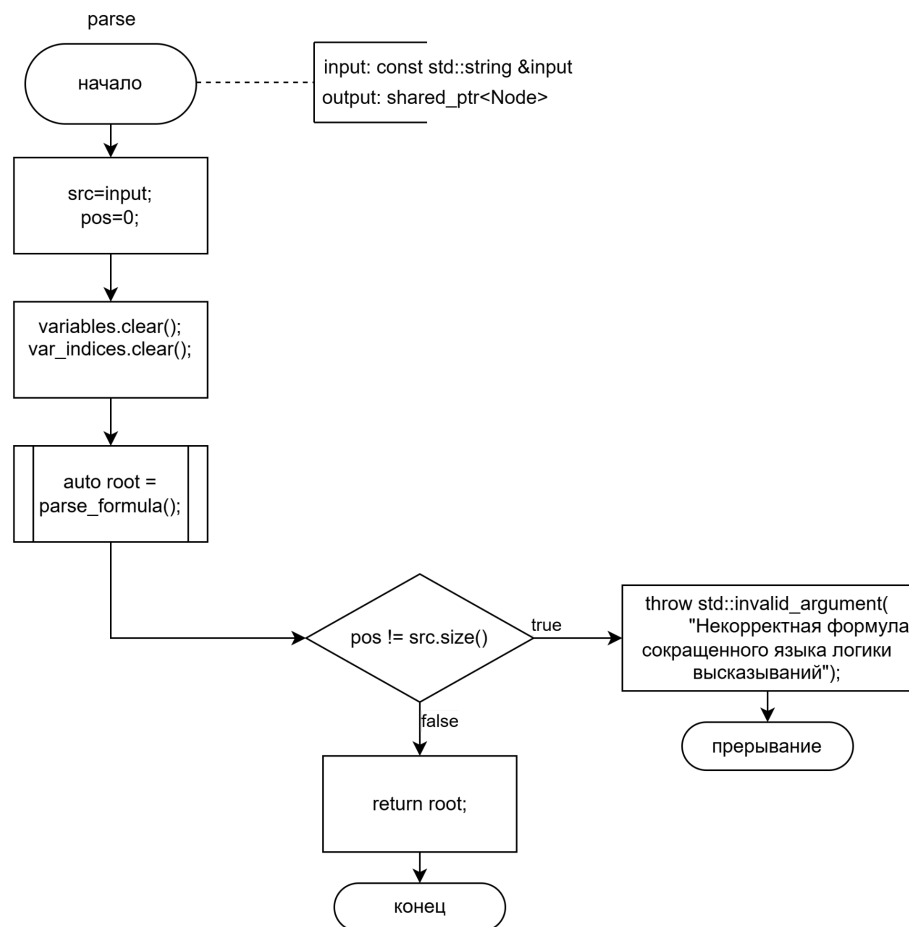


Рис. 3.5 - Parser::parse()

Функция `Parser::parse_formula()` позволяет построить дерево по формуле. Для удобства прочтения схема была поделена на несколько частей:

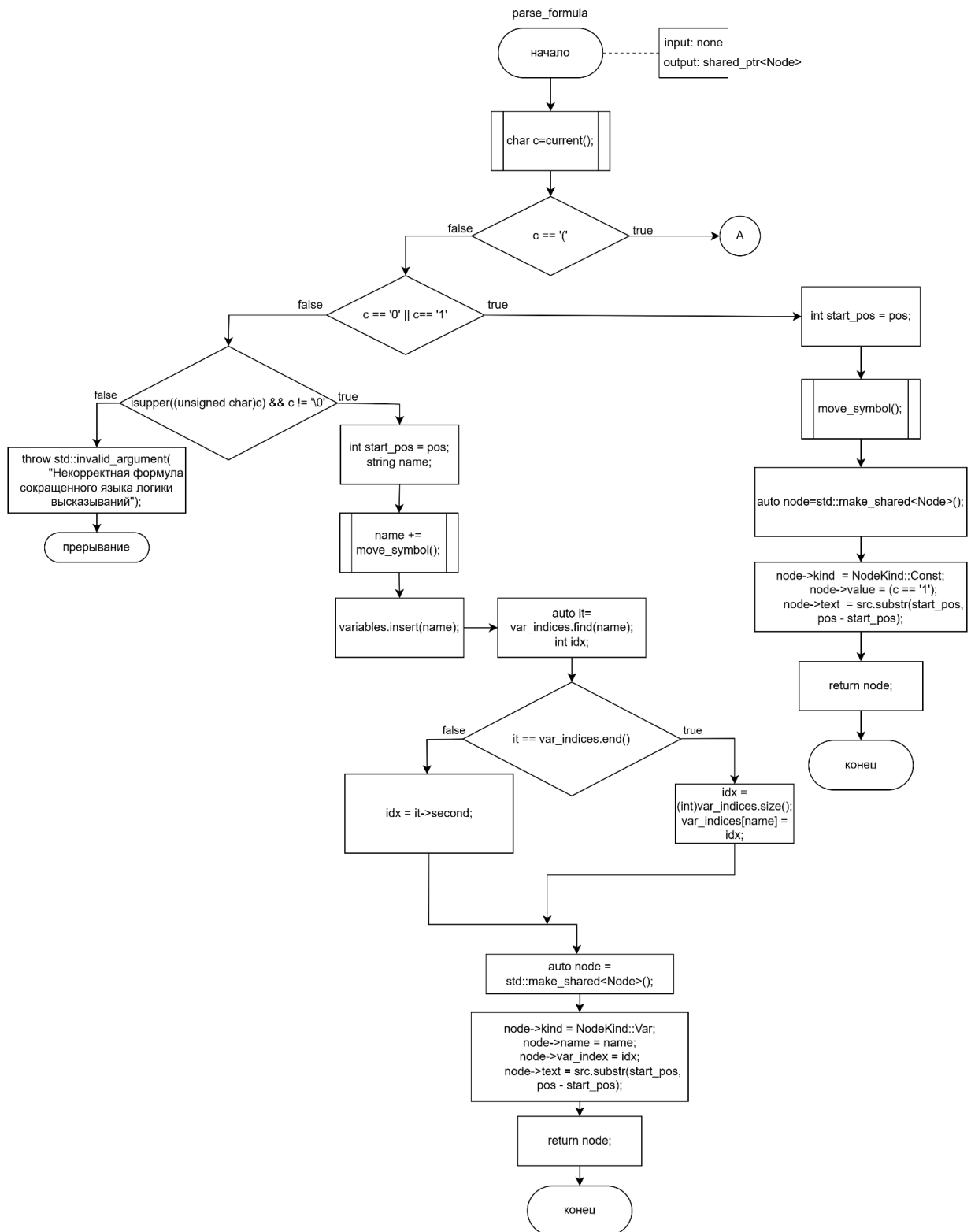


Рис. 3.6 - `Parser::parse_formula()`

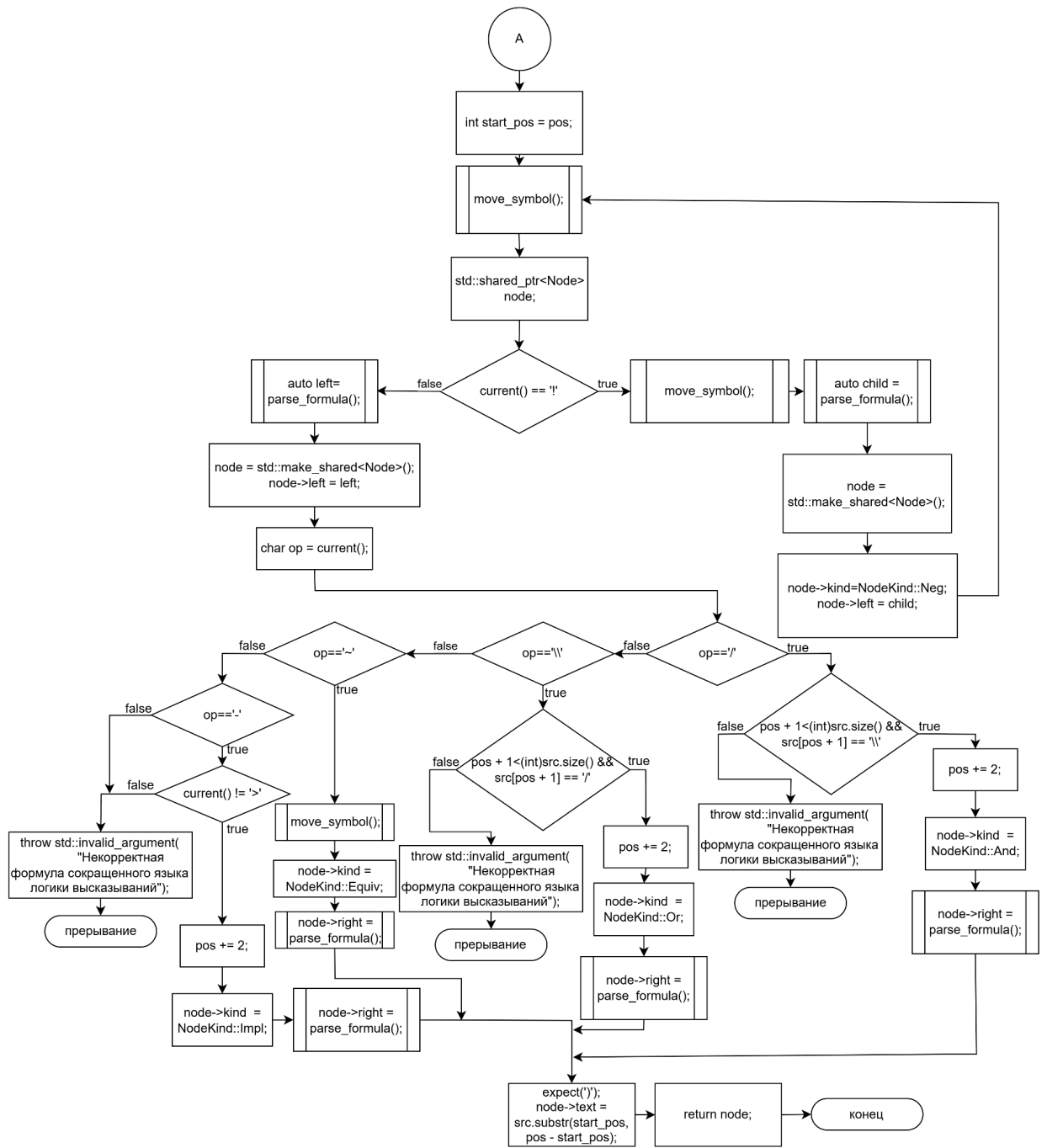


Рис. 3.7 - Parser::parse_formula()

4. Тестирование программы

При выполнении пользователя встречает окно с возможностью ввода формулы:

```
E:\LABS_FOURTH_SEM\LOIS\lab1\cmake-build-debug\lab1.exe
Формула:
```

В случае неверного ввода формулы будет выведено сообщение с ошибкой:

```
Формула: (!A
Ошибка синтаксиса: Некорректная формула сокращенного языка логики высказываний
```

Иначе программа начнет выполнение:

```
Формула: (A\B)

A | B | (A\B)
0 | 0 | 0
0 | 1 | 1
...

Формула является ВЫПОЛНИМОЙ.

Формула: (P\(!P))

P | (P\(!P))
0 | 0
1 | 0

Формула является НЕВЫПОЛНИМОЙ.
```

Максимальный предел количества переменных для построения в консоли таблицы истинности - 15. Для 15 переменных в худшем случае время работы

программы составило 203 мс (Windows 11 Pro 25H2, Intel(R) Core(TM) i5 125H (2.50 GHz), ОЗУ:16гб, ПЗУ:1024гб, режим работы: сбалансированный):

Тестовый случай:

$(A \wedge (B \wedge (C \wedge (D \wedge (E \wedge (F \wedge (G \wedge (H \wedge (I \wedge (J \wedge (K \wedge (L \wedge (M \wedge (N \wedge (O \wedge (!A))))))))))))))$

1	1	1	1	1	1	1	1	1	1	1	1	0	1	0	1	0
1	1	1	1	1	1	1	1	1	1	1	1	0	1	1	0	0
1	1	1	1	1	1	1	1	1	1	1	1	0	1	1	1	0
1	1	1	1	1	1	1	1	1	1	1	1	1	0	0	0	0
1	1	1	1	1	1	1	1	1	1	1	1	1	0	0	1	0
1	1	1	1	1	1	1	1	1	1	1	1	1	0	1	0	0
1	1	1	1	1	1	1	1	1	1	1	1	1	0	1	1	0
1	1	1	1	1	1	1	1	1	1	1	1	1	1	0	0	0
1	1	1	1	1	1	1	1	1	1	1	1	1	1	0	1	0
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	0	0
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	0

Время выполнения: 203 мс

Формула является НЕВЫПОЛНИМОЙ.

При большом количестве переменных (>15), таблица истинности не строится.

Тестовый случай:

$(A \wedge (B \wedge (C \wedge (D \wedge (E \wedge (F \wedge (G \wedge (H \wedge (I \wedge (J \wedge (K \wedge (L \wedge (M \wedge (N \wedge (O \wedge (P \wedge (Q \wedge (R \wedge (S \wedge (T \wedge (!A))))))))))))))))$

Результат:

Формула: $A \wedge (B \wedge (C \wedge (D \wedge (E \wedge (F \wedge (G \wedge (H \wedge (I \wedge (J \wedge (K \wedge (L \wedge (M \wedge (N \wedge (O \wedge (P \wedge (Q \wedge (R \wedge (S \wedge (T \wedge (!A))))))))))))))$
Слишком много переменных (20) – таблица не выводится, только проверка.
Время выполнения: 490 мс

Время работы для 20 переменных составило 490 мс (Windows 11 Pro 25H2, Intel(R) Core(TM) i5 125H (2.50 GHz), ОЗУ:16гб, ПЗУ:1024гб, режим работы: сбалансированный)

Тестовый случай:

$(A \wedge (B \wedge (C \wedge (D \wedge (E \wedge (F \wedge (G \wedge (H \wedge (I \wedge (J \wedge (K \wedge (L \wedge (M \wedge (N \wedge (O \wedge (P \wedge (Q \wedge (R \wedge (S \wedge (T \wedge (U \wedge (V \wedge (W \wedge (X \wedge Y))))))))))))))))))))))$

Результат:

```
Формула: (A/\(B/\(C/\(D/\(E/\(F/\(G/\(H/\(I/\(J/\(K/\(L/\(M/\(N/\(O/\(P/\(Q/\(R/\(S/\(T/\(U/\(V/\(W/\(X/\(Y))))))))))))))))))
Слишком много переменных (25) – таблица не выводится, только проверка.
Время выполнения: 23571 мс
Формула является ВЫПОЛНИМОЙ.
```

Время работы для 25 переменных составило 23571 мс(Windows 11 Pro 25H2, Intel(R) Core(TM) i5 125H (2.50 GHz), ОЗУ:16гб, ПЗУ:1024гб, режим работы: сбалансированный)

Вывод

В результате выполнения лабораторной работы были приобретены навыки программирования алгоритмов синтаксического разбора формул сокращенного языка логики высказываний. Также была реализована процедурная программа синтаксического разбора формул сокращенного языка логики высказываний, проверяющая, является ли формула невыполнимой.

Список использованных источников

1. Голенков, В. В. Логические основы интеллектуальных систем. Практикум : учеб.-метод. пособие / В. В. Голенков [и др.]. – Минск : БГУИР, 2011. – 70 с.
2. Метод рекурсивного спуска - URL: [https://ru.wikibooks.org/Реализации_алгоритмов/Метод рекурсивного спуска](https://ru.wikibooks.org/Реализации_алгоритмов/Метод_рекурсивного_спуска) (дата доступа: 03.04.2026)
3. cppreference.com: C++ Reference [Электронный ресурс] : cppreference.com. – Режим доступа: <https://cppreference.com/w/cpp/algorithm.html>. – (дата доступа: 03.04.2026.)

Приложение А

Листинг программы

```
/*  
  
 * Лабораторная работа №1 по дисциплине "Логические основы  
интеллектуальных систем"  
  
 * Выполнена студентом группы 421702 БГУИР Евик Алексей  
Николаевич  
  
 */  
  
/*  
  
 * Файл node.h.  
  
 * Описывает один узел дерева формулы и типы операций  
  
 * (переменная, константа, отрицание, конъюнкция, дизъюнкция,  
импликация, эквиваленция).  
  
 * Использованные материалы - Логические основы интеллектуальных  
систем. Практикум: учеб.-метод. пособие /  
  
 * В. В. Голенков [и др.]. - Минск: БГУИР, 2011. - 70 с.  
  
 * Алгоритм - Метод рекурсивного спуска - URL:  
https://ru.wikibooks.org/Реализации\_алгоритмов/Метод  
рекурсивного спуска  
  
 * (дата обращения: 03.04.2026)  
  
 */  
  
#ifndef LAB1_NODE_H  
  
#define LAB1_NODE_H  
  
#include <string>  
  
#include <memory>  
  
#include <vector>  
  
enum class NodeKind {  
  
    Var,  
  
    Const,  
  
    Neg,
```



```

    And,

    Or,

    Impl,

    Equiv
};

struct Node {
    NodeKind kind;

    std::string name;

    std::string text;

    bool value = false;

    int var_index = -1;

    std::shared_ptr<Node> left;

    std::shared_ptr<Node> right;

    bool eval(const std::vector<bool>& interp) const;
};

#endif //LAB1_NODE_H

/*

* Лабораторная работа №1 по дисциплине "Логические основы
интеллектуальных систем"

* Выполнена студентом группы 421702 БГУИР Евик Алексей
Николаевич

*/

/*

* Содержит рекурсивную функцию для вычисления значения формулы
при заданной интерпретации переменных.

* Используемые материалы - Логические основы интеллектуальных
систем. Практикум: учеб.-метод. пособие /

* В. В. Голенков [и др.]. - Минск: БГУИР, 2011. - 70 с.

```

* Алгоритм - Метод рекурсивного спуска - URL:
[https://ru.wikibooks.org/Реализации_алгоритмов/Метод
рекурсивного спуска](https://ru.wikibooks.org/Реализации_алгоритмов/Метод_рекурсивного_спуска)

* (дата обращения: 03.04.2026)

*/

#include "node.h"

```
bool Node::eval(const std::vector<bool>& interp) const {  
    switch (kind) {  
        case NodeKind::Var:  
            return interp[var_index];  
        case NodeKind::Const:  
            return value;  
        case NodeKind::Neg:  
            return !left->eval(interp);  
        case NodeKind::Or:  
            return left->eval(interp) || right->eval(interp);  
        case NodeKind::And:  
            return left->eval(interp) && right->eval(interp);  
        case NodeKind::Equiv:  
            return left->eval(interp) == right->eval(interp);  
        case NodeKind::Impl:  
            return !left->eval(interp) || right->eval(interp);  
    }  
    return false;  
}  
/*
```

* Лабораторная работа №1 по дисциплине "Логические основы
интеллектуальных систем"

* Выполнена студентом группы 421702 БГУИР Евик Алексей Николаевич

*/

/*

* Объявляет интерфейс синтаксического анализатора:

* метод разбора строки в дерево и вспомогательные методы для работы с позицией чтения.

* Использованные материалы - Логические основы интеллектуальных систем. Практикум: учеб.-метод. пособие /

* В. В. Голенков [и др.]. - Минск: БГУИР, 2011. - 70 с.

* Алгоритм - Метод рекурсивного спуска - URL:
[https://ru.wikibooks.org/Реализации_алгоритмов/Метод рекурсивного спуска](https://ru.wikibooks.org/Реализации_алгоритмов/Метод_рекурсивного_спуска)

* (дата обращения: 03.04.2026)

*/

```
#ifndef LAB1_PARSER_H
```

```
#define LAB1_PARSER_H
```

```
#include <set>
```

```
#include <string>
```

```
#include <memory>
```

```
#include <node/node.h>
```

```
#include <map>
```

```
class Parser {
```

```
public:
```

```
    std::set<std::string> variables;
```

```
    std::map<std::string, int> var_indices;
```

```
    std::shared_ptr<Node> parse(const std::string& input);
```

```
private:
```

```
    std::string src;
```

```
    int pos = 0;
```

```
    char current();
```

```

char move_symbol();

void expect(char c);

std::shared_ptr<Node> parse_formula();

};

#endif //LAB1_PARSER_H

/*
 * Лабораторная работа №1 по дисциплине "Логические основы
интеллектуальных систем"

 * Выполнена студентом группы 421702 БГУИР Евик Алексей
Николаевич

 */

/*
 * Содержит алгоритм рекурсивного спуска для разбора формул
сокращённого языка логики высказываний и построения

 * абстрактного дерева.

 * Используемые материалы - Логические основы интеллектуальных
систем. Практикум: учеб.-метод. пособие /

 * В. В. Голенков [и др.]. - Минск: БГУИР, 2011. - 70 с.

 * Алгоритм - Метод рекурсивного спуска - URL:
https://ru.wikibooks.org/Реализации\_алгоритмов/Метод
рекурсивного спуска

 * (дата обращения: 03.04.2026)

 */

#include "parser.h"

#include <memory>

/*Реализует обработку формулы сокращенного языка логики
высказываний*/

std::shared_ptr<Node> Parser::parse(const std::string &input) {

    src = input;

    pos = 0;

    variables.clear();

```

```

    var_indices.clear();

    auto root = parse_formula();

    if (pos != (int)src.size()) {

        throw std::invalid_argument(

            "Некорректная формула сокращенного языка логики
высказываний");

    }

    return root;
}

/*Позволяет обратиться к текущему символу выражения*/
char Parser::current() {

    return (pos < (int)src.size()) ? src[pos] : '\0';

}

/*Позволяет перейти на символ вперед*/
char Parser::move_symbol() {

    if (pos >= (int)src.size()) {

        throw std::invalid_argument("Неожиданный конец строки");

    }

    return src[pos++];

}

/*Ожидает на выходе определенный символ, если он не совпадает с
ожидаемым - выбрасывается ошибка*/
void Parser::expect(char c) {

    if (current() != c) {

        throw std::invalid_argument("Некорректная формула
сокращенного языка логики высказываний");

    }

    move_symbol();

```

```
}
```

```
/*Позволяет построить дерево по формуле для корректной  
обработки*/
```

```
std::shared_ptr<Node> Parser::parse_formula() {
```

```
    char c = current();
```

```
    /*Ожидание открывающей скобки*/
```

```
    if (c == '(') {
```

```
        int start_pos = pos;
```

```
        move_symbol();
```

```
        std::shared_ptr<Node> node;
```

```
        /*Отрицание*/
```

```
        if (current() == '!') {
```

```
            move_symbol();
```

```
            auto child = parse_formula();
```

```
            node = std::make_shared<Node>();
```

```
            node->kind = NodeKind::Neg;
```

```
            node->left = child;
```

```
        }
```

```
    else {
```

```
        auto left = parse_formula();
```

```
        node = std::make_shared<Node>();
```

```
        node->left = left;
```

```
        switch (current()) {
```

```
            /*Конъюнкция*/
```

```
            case '/':
```

```
                if (pos + 1 < (int)src.size() && src[pos +  
1] == '\\') {
```

```

        pos += 2;

        node->kind = NodeKind::And;

        node->right = parse_formula();

    } else {

        throw std::invalid_argument(

            "Некорректная формула сокращенного
языка логики высказываний");

    }

    break;

    /*Дизъюнкция*/

case '\\':

    if (pos + 1 < (int)src.size() && src[pos +
1] == '/') {

        pos += 2;

        node->kind = NodeKind::Or;

        node->right = parse_formula();

    } else {

        throw std::invalid_argument(

            "Некорректная формула сокращенного
языка логики высказываний");

    }

    break;

case '~':

    /*Эквиваленция*/

    move_symbol();

    node->kind = NodeKind::Equiv;

    node->right = parse_formula();

    break;

    /*Импликация*/

case '-':

```

```

        move_symbol();

        if (current() != '>') {

            throw
std::invalid_argument("Некорректная формула сокращенного языка
логики высказываний");

        }

        move_symbol();

        node->kind = NodeKind::Impl;

        node->right = parse_formula();

        break;

    default:

        throw std::invalid_argument(
"Некорректная формула сокращенного языка логики
высказываний");

    }

}

expect(')');

node->text = src.substr(start_pos, pos - start_pos);

return node;

}

/*Логические константы*/

if (c == '0' || c == '1') {

    int start_pos = pos;

    move_symbol();

    auto node    = std::make_shared<Node>();

    node->kind    = NodeKind::Const;

    node->value   = (c == '1');

    node->text    = src.substr(start_pos, pos - start_pos);

    return node;

}

```



```

    if (std::isupper((unsigned char)c) && c != '\\0') {
        int start_pos = pos;
        std::string name;
        name += move_symbol();
        variables.insert(name);
        auto it = var_indices.find(name);
        int idx;
        if (it == var_indices.end()) {
            idx = (int)var_indices.size();
            var_indices[name] = idx;
        } else {
            idx = it->second;
        }

        auto node = std::make_shared<Node>();
        node->kind = NodeKind::Var;
        node->name = name;
        node->var_index = idx;
        node->text = src.substr(start_pos, pos - start_pos);
        return node;
    }

    throw std::invalid_argument(
        "Некорректная формула сокращенного языка логики
        высказываний");
}

/*
    * Лабораторная работа №1 по дисциплине "Логические основы
    интеллектуальных систем"

```

* Выполнена студентом группы 421702 БГУИР Евик Алексей Николаевич

* Точка входа программы.

* Содержит главный цикл взаимодействия с пользователем,

* обработку ввода формулы, построение таблицы истинности и вывод результата проверки на выполнимость.

* Использованные материалы - Логические основы интеллектуальных систем. Практикум: учеб.-метод. пособие /

* В. В. Голенков [и др.]. - Минск: БГУИР, 2011. - 70 с.

* Алгоритм - Метод рекурсивного спуска - URL:
[https://ru.wikibooks.org/Реализации_алгоритмов/Метод рекурсивного спуска](https://ru.wikibooks.org/Реализации_алгоритмов/Метод_рекурсивного_спуска)

* (дата обращения: 03.04.2026)

*/

```
#include <node/node.h>
```

```
#include <parser/parser.h>
```

```
#include <vector>
```

```
#include <iostream>
```

```
#include <sstream>
```

```
#include <stdexcept>
```

```
#include <memory>
```

```
#include <windows.h>
```

```
#include <chrono>
```

```
using namespace std;
```

```
bool build_truth_table(const shared_ptr<Node>& root,
```

```
    const set<string>& var_set,
```

```
    const map<string, int>& var_indices,
```

```
    const string& formula_text) {
```

```
    vector vars(var_set.begin(), var_set.end());
```

```

int n = (int)vars.size();
long long rows = (n == 0) ? 1 : (1LL << n);
bool print_table = (n <= 15);
vector<int> bit_to_var(n);
for (int i = 0; i < n; ++i) {
    bit_to_var[i] = var_indices.at(vars[i]);
}

ostringstream buf;

if (print_table) {
    for (auto& v : vars)
        buf << v << " | ";
    buf << formula_text << "\n";
} else {
    cout << "Слишком много переменных (" << n
        << ") — таблица не выводится, только проверка.\n";
}

bool any_true = false;
vector<bool> interp(n); // <-- вектор вместо map

for (long long mask = 0; mask < rows; ++mask) {
    for (int i = 0; i < n; ++i) {
        bool val = (mask >> (n - 1 - i)) & 1;
        interp[bit_to_var[i]] = val;
    }
}

```

```

        bool result = root->eval(interp);

        if (print_table) {
            for (int i = 0; i < n; ++i)
                buf << (interp[bit_to_var[i]] ? '1' : '0') << "
| ";

            buf << (result ? '1' : '0') << '\n';
        }

        if (result) {
            any_true = true;
            if (print_table) buf << "...\\n";
            break;
        }
    }

    if (print_table) cout << buf.str();
    return !any_true;
}

int main() {
#ifdef _WIN32
    SetConsoleOutputCP(65001);
    SetConsoleCP(65001);
#endif

    while (true) {
        cout << "Формула: ";
        string formula;
        if (!getline(cin, formula)) break;
        if (formula.empty()) continue;
    }
}

```

```

        if (formula == "exit") break;

        Parser parser;

        shared_ptr<Node> root;

        try {

            root = parser.parse(formula);

        }

        catch (const invalid_argument& ex) {

            cout << "Ошибка синтаксиса: " << ex.what() <<
"\n\n";

            continue;

        }

        bool unsat = false;

        try {

            auto start = chrono::steady_clock::now();

            unsat = build_truth_table(root, parser.variables,
parser.var_indices, formula);

            auto end = chrono::steady_clock::now();

            auto ms =
chrono::duration_cast<chrono::milliseconds>(end -
start).count();

            cout << "\nВремя выполнения: " << ms << " мс\n";

        }

        catch (const exception& ex) {

            cout << "Ошибка вычисления: " << ex.what() <<
"\n\n";

            continue;

        }

```

```
    cout << "\nФормула является ";  
    if (unsat)  
        cout << "НЕВЫПОЛНИМОЙ.\n";  
    else  
        cout << "ВЫПОЛНИМОЙ.\n";  
    cout << "\n";  
}  
return 0;  
}
```